

Spring Boot Application Flow

1. Introduction

A Spring Boot application follows a layered architecture similar to Spring Framework, but Spring Boot makes application development and execution easier through features such as:

- Auto Configuration
- Starter Dependencies
- Embedded Server
- Spring Boot configuration
- Spring Data JPA integration
- Production-ready features
- Simplified application startup

A typical REST API request follows this flow:

```
Client
  ↓
Embedded Server (Tomcat)
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

The response travels back in the reverse direction:

```
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
  ↓
DispatcherServlet
  ↓
```

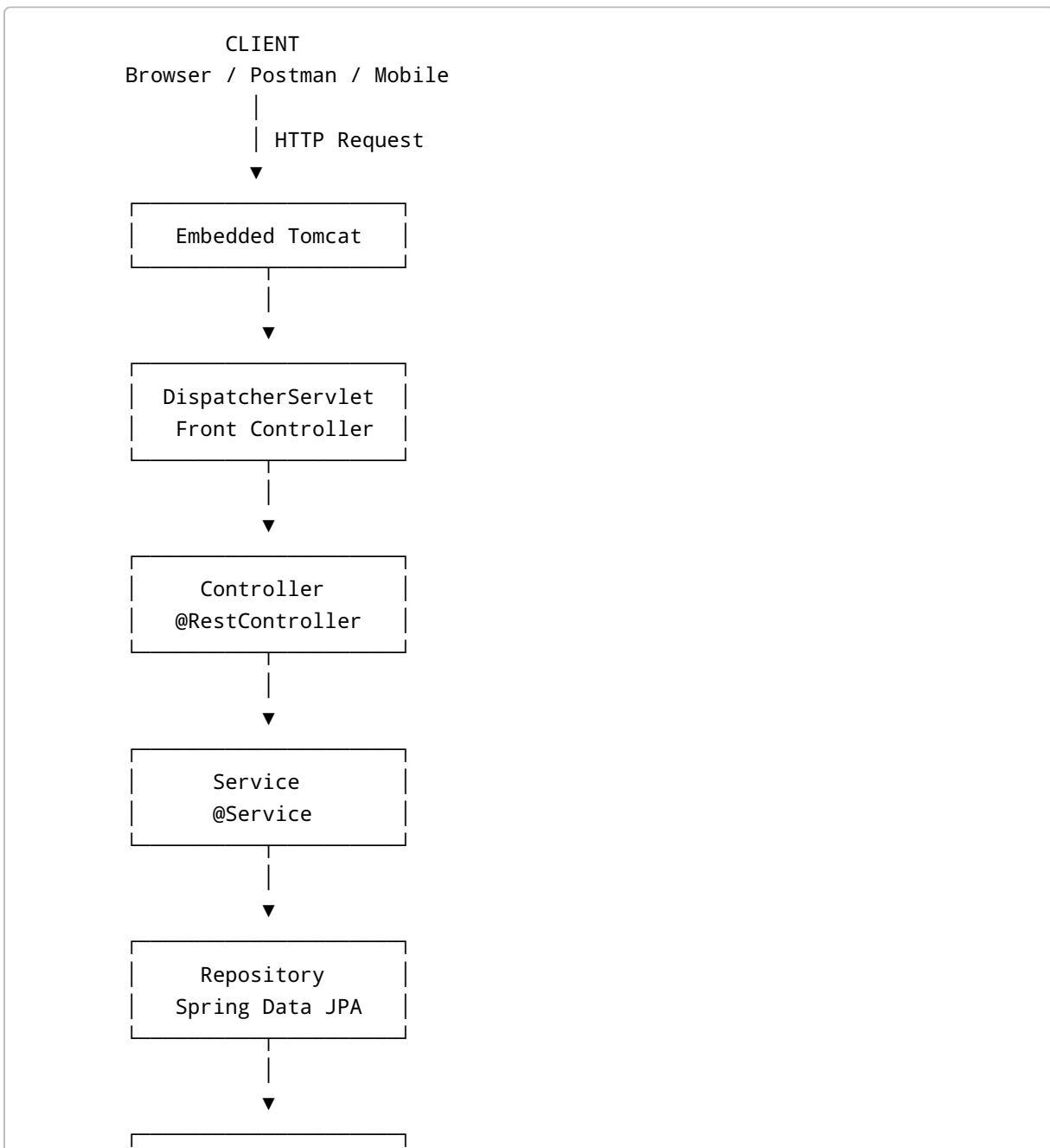
Embedded Server



Client

2. High-Level Spring Boot Architecture

A typical Spring Boot REST application contains:



Database PostgreSQL / MySQL

3. What Happens When We Start a Spring Boot Application?

A Spring Boot application usually starts from the `main()` method.

Example:

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            EmployeeApplication.class,
            args
        );
    }
}
```

The important line is:

```
SpringApplication.run(EmployeeApplication.class, args);
```

4. Spring Boot Startup Flow

When we execute the application:

```
main()
↓
SpringApplication.run()
↓
Spring Boot Starts
↓
```

```
ApplicationContext Created
↓
Auto Configuration Applied
↓
Component Scanning
↓
Beans Created
↓
Dependencies Injected
↓
Embedded Tomcat Starts
↓
Application Ready
```

For example:

```
EmployeeApplication
↓
SpringApplication.run()
↓
Spring Container
↓
Controller Bean
Service Bean
Repository Bean
DataSource Bean
↓
Embedded Tomcat
↓
Ready to Accept Requests
```

5. @SpringBootApplication

The main Spring Boot application class normally contains:

```
@SpringBootApplication
```

Example:

```
@SpringBootApplication
public class EmployeeApplication {
}
```

This annotation combines three important concepts:

```
@SpringBootApplication
|
|— @SpringBootConfiguration
|
|— @EnableAutoConfiguration
|
|— @ComponentScan
```

6. Component Scanning

Spring Boot scans the application packages for Spring components.

For example:

```
com.javacodeex.employee
|
|— EmployeeApplication.java
|
|— controller
|   |— EmployeeController.java
|
|— service
|   |— EmployeeService.java
|
|— repository
|   |— EmployeeRepository.java
|
|— entity
|   |— Employee.java
```

Spring discovers classes annotated with:

```
@Component
@Service
```

```
@Repository
@Controller
@RestController
@Configuration
```

Spring then creates and manages the required objects.

7. Spring Container

The Spring Container manages application objects.

These managed objects are called:

Spring Beans

For example:

```
Spring Container
|
├─ EmployeeController Bean
├─ EmployeeService Bean
├─ EmployeeRepository Bean
├─ DataSource Bean
└─ Other Beans
```

The container is also responsible for injecting dependencies.

8. Dependency Injection

Consider:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(
        EmployeeService employeeService) {
```

```
        this.employeeService = employeeService;
    }
}
```

The controller needs `EmployeeService`.

We don't write:

```
EmployeeService service =
    new EmployeeService();
```

Instead, Spring creates the `EmployeeService` bean and injects it into the controller.

Similarly:

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }
}
```

Spring injects the repository.

Therefore:

```
Controller
  ↓
Service
  ↓
Repository
```

is wired by the Spring Container.

9. Embedded Server

One of Spring Boot's important features is its support for an **embedded server**.

For a typical Spring Boot web application, the default server is:

Tomcat

Instead of installing Tomcat separately:

Traditional Approach

Install Tomcat

↓

Configure Tomcat

↓

Build WAR

↓

Deploy WAR

Spring Boot commonly uses:

Spring Boot

Build Application

↓

Embedded Tomcat

↓

Run JAR

Example:

```
java -jar employee-service.jar
```

10. Client Sends HTTP Request

Once the application starts, clients can send HTTP requests.

Clients could be:

- Browser
- Postman
- Angular
- React
- Mobile application
- Another microservice

Example:

```
GET /employees/101
```

Complete URL:

```
http://localhost:8080/employees/101
```

Flow begins:

```
Client
  ↓
HTTP Request
  ↓
Embedded Tomcat
```

11. Embedded Tomcat Receives the Request

Tomcat listens for incoming HTTP requests.

Default Spring Boot port:

```
8080
```

So:

```
http://localhost:8080
```

Tomcat receives:

```
GET /employees/101
```

and passes the request into the web application.

12. DispatcherServlet

The request reaches:

DispatcherServlet

DispatcherServlet is the Front Controller of Spring MVC.

Flow:

```
Client
  ↓
Tomcat
  ↓
DispatcherServlet
```

Its responsibilities include:

- Receiving HTTP requests
 - Determining the appropriate controller
 - Routing requests
 - Coordinating Spring MVC request processing
 - Returning the HTTP response
-

13. Controller Mapping

Suppose we have:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @GetMapping("/{id}")
    public EmployeeResponseDto getEmployee(
        @PathVariable Long id) {
```

```
        // ...  
    }  
}
```

For:

```
GET /employees/101
```

Spring determines that:

```
@GetMapping("/{id}")
```

is the matching handler.

Flow:

```
DispatcherServlet  
    ↓  
EmployeeController  
    ↓  
getEmployee(101)
```

14. Controller Layer

The Controller handles HTTP-related operations.

Example:

```
@RestController  
@RequestMapping("/employees")  
public class EmployeeController {  
  
    private final EmployeeService employeeService;  
  
    public EmployeeController(  
        EmployeeService employeeService) {  
  
        this.employeeService = employeeService;  
    }  
}
```

```

@GetMapping("/{id}")
public ResponseEntity<EmployeeResponseDto>
    getEmployee(@PathVariable Long id) {

    EmployeeResponseDto response =
        employeeService.getEmployee(id);

    return ResponseEntity.ok(response);
}
}

```

Controller responsibilities:

- Receive request
- Read request body
- Read path variables
- Read query parameters
- Trigger validation
- Call service
- Prepare HTTP response

15. Service Layer

The Controller calls the Service.

Example:

```
employeeService.getEmployee(id);
```

Service:

```

@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }
}

```

```
public EmployeeResponseDto getEmployee(Long id) {  
  
    Employee employee =  
        employeeRepository.findById(id)  
            .orElseThrow();  
  
    return new EmployeeResponseDto(  
        employee.getId(),  
        employee.getName(),  
        employee.getEmail()  
    );  
}  
}
```

16. Service Layer Responsibilities

The Service layer handles:

- Business logic
- Business validations
- Calculations
- Transaction management
- Data transformation
- Calling repositories
- Calling other services
- Entity-to-DTO conversion

Example:

Controller

"What HTTP request came?"

↓

Service

"What business operation should happen?"

17. Repository Layer

The Service communicates with the Repository.

Example:

```
employeeRepository.findById(id);
```

Repository:

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
}
```

With Spring Data JPA, we don't need to implement common CRUD operations manually.

18. Common Repository Methods

`JpaRepository` provides methods such as:

```
save()
findById()
findAll()
deleteById()
existsById()
count()
```

For example:

```
employeeRepository.findById(101L);
```

Spring Data JPA performs the required database operation.

19. Database Layer

Repository communicates with the configured database.

Examples:

- PostgreSQL
- MySQL
- Oracle
- SQL Server

Flow:

```
Repository
  ↓
JPA / Hibernate
  ↓
JDBC
  ↓
Database
```

Suppose:

```
employeeRepository.findById(101L);
```

Conceptually, a query similar to this may be executed:

```
SELECT *
FROM employees
WHERE id = 101;
```

20. Database Returns Data

Suppose the database contains:

```
ID      : 101
Name    : Anand
Email   : anand@example.com
```

The data travels back:

```
Database
  ↓
```

Hibernate / JPA

↓

Repository

↓

Service

21. Service Processes the Data

The Service can convert the Entity into a Response DTO.

Entity:

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;
}
```

Response DTO:

```
public record EmployeeResponseDto(
    Long id,
    String name,
    String email) {
}
```

Service converts:

```
Employee Entity
    ↓
EmployeeResponseDto
```


22. Controller Returns Response

Service returns:

```
EmployeeResponseDto
```

to the Controller.

Controller:

```
return ResponseEntity.ok(response);
```

Response status:

```
200 OK
```

23. Java Object to JSON

For REST APIs, Spring Boot commonly uses Jackson to serialize Java objects into JSON.

Java:

```
new EmployeeResponseDto(  
    101L,  
    "Anand",  
    "anand@example.com"  
);
```

becomes:

```
{  
  "id": 101,  
  "name": "Anand",  
  "email": "anand@example.com"  
}
```

24. Complete GET Request Flow

For:

GET /employees/101

the complete flow is:

```
1. Client
   ↓
2. Embedded Tomcat
   ↓
3. DispatcherServlet
   ↓
4. EmployeeController
   ↓
5. EmployeeService
   ↓
6. EmployeeRepository
   ↓
7. JPA / Hibernate
   ↓
8. Database
```

Response:

```
Database
   ↓
Repository
   ↓
Service
   ↓
Controller
   ↓
DispatcherServlet
   ↓
Tomcat
   ↓
Client
```

25. POST Request Example

Suppose the client wants to create an employee.

Request:

```
POST /employees
```

Body:

```
{
  "name": "Anand",
  "email": "anand@example.com"
}
```

Flow:

```
Postman
  ↓
Tomcat
  ↓
DispatcherServlet
  ↓
EmployeeController
  ↓
EmployeeService
  ↓
EmployeeRepository
  ↓
PostgreSQL
```

26. Controller for POST

```
@PostMapping
public ResponseEntity<EmployeeResponseDto>
    createEmployee(
        @RequestBody EmployeeRequestDto request) {

    EmployeeResponseDto response =
```

```
        employeeService.createEmployee(request);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(response);
}
```

27. Service for POST

```
public EmployeeResponseDto createEmployee(
    EmployeeRequestDto request) {

    Employee employee = new Employee();

    employee.setName(request.name());
    employee.setEmail(request.email());

    Employee saved =
        employeeRepository.save(employee);

    return new EmployeeResponseDto(
        saved.getId(),
        saved.getName(),
        saved.getEmail()
    );
}
```

28. Repository Saves Data

```
employeeRepository.save(employee);
```

Spring Data JPA and Hibernate handle the persistence operation.

Conceptually:

```
save(employee)
  ↓
Spring Data JPA
  ↓
```

```
Hibernate
  ↓
JDBC
  ↓
INSERT Query
  ↓
Database
```

29. Auto Configuration

One of the biggest differences between traditional Spring and Spring Boot is:

Auto Configuration

Spring Boot examines the dependencies and configuration available in the project and configures many components automatically.

For example, when we add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-web
  </artifactId>
</dependency>
```

Spring Boot can configure:

- Spring MVC
- DispatcherServlet
- Jackson
- Embedded Tomcat
- HTTP message converters

30. Spring Data JPA Auto Configuration

When we add:

```
spring-boot-starter-data-jpa
```

Spring Boot can configure infrastructure for:

```
DataSource
  ↓
JPA
  ↓
Hibernate
  ↓
EntityManager
  ↓
Transaction Management
```

We mainly provide the database connection details.

31. application.properties

Spring Boot allows configuration through:

```
application.properties
```

Example:

```
server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/employeedb
spring.datasource.username=postgres
spring.datasource.password=admin

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

32. application.yml

The same configuration can be written as YAML:

```
server:
  port: 8080
```

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/employeeedb
    username: postgres
    password: admin

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

33. How Auto Configuration Works Conceptually

```
Spring Boot Starts
  ↓
Checks Classpath
  ↓
Checks Dependencies
  ↓
Checks Existing Beans
  ↓
Checks Configuration Properties
  ↓
Applies Suitable Auto Configuration
```

For example:

```
Spring Web dependency found
  ↓
Configure MVC
  ↓
Configure DispatcherServlet
  ↓
Configure Embedded Server
```

34. Starter Dependencies

Spring Boot starters group commonly required dependencies.

Examples:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
spring-boot-starter-validation
spring-boot-starter-test
spring-boot-starter-actuator
```

Instead of manually managing many individual dependencies, we select the required starter.

35. Transaction Management

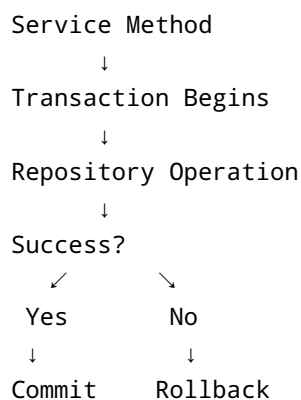
Transactions are usually defined at the Service layer.

Example:

```
@Transactional
public EmployeeResponseDto createEmployee(
    EmployeeRequestDto request) {

    // business logic
}
```

Flow:



36. Validation Flow

Suppose the request DTO is:

```
public record EmployeeRequestDto(  
  
    @NotBlank  
    String name,  
  
    @Email  
    String email) {  
  
}
```

Controller:

```
@PostMapping  
public ResponseEntity<EmployeeResponseDto>  
    createEmployee(  
        @Valid @RequestBody EmployeeRequestDto request) {  
  
    // ...  
  
}
```

Flow:

```
HTTP Request  
  ↓  
Controller  
  ↓  
Validation  
  ↙   ↘  
Valid   Invalid  
  ↓     ↓  
Service Error Response
```

37. Exception Handling

Spring Boot applications commonly use centralized exception handling.

Example:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<String> handleNotFound(
        EmployeeNotFoundException exception) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(exception.getMessage());
    }
}

```

Flow:

```

Service / Repository
    ↓
Exception
    ↓
@RestControllerAdvice
    ↓
Error Response
    ↓
Client

```

38. Security Flow

If Spring Security is used:

```

Client
    ↓
Spring Security Filter Chain
    ↓
Embedded Server / MVC Processing
    ↓
DispatcherServlet
    ↓
Controller

```

Security can handle:

- Authentication
 - Authorization
 - JWT validation
 - Roles and permissions
-

39. Actuator

Spring Boot Actuator provides production-ready monitoring features.

Dependency:

```
spring-boot-starter-actuator
```

Example endpoint:

```
/actuator/health
```

Response:

```
{
  "status": "UP"
}
```

Actuator can provide:

- Health information
- Metrics
- Application information
- Environment details
- Bean information

depending on configuration.

40. Traditional Spring vs Spring Boot Flow

The actual MVC request flow is largely based on Spring MVC in both.

The main difference is **how the application infrastructure is configured and started**.

Traditional Spring

```
Manual Configuration
↓
External Server commonly used
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
```

Spring Boot

```
Auto Configuration
↓
Embedded Server
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
```

41. Important Point

Students sometimes think Spring Boot has a completely different request-processing architecture from Spring.

That is incorrect.

Spring Boot commonly uses Spring Framework components internally.

For example:

```
Spring Boot
↓
Spring Framework
↓
Spring MVC
↓
DispatcherServlet
```

Spring Boot mainly simplifies configuration, startup, dependency management, and deployment.

42. Responsibilities of Each Layer

Component	Responsibility
Client	Sends HTTP request
Embedded Server	Receives network request
DispatcherServlet	Front Controller
Controller	Handles API request
Service	Handles business logic
Repository	Handles data access
JPA/Hibernate	ORM/persistence
Database	Stores application data
Spring Container	Manages beans and DI
Auto Configuration	Configures infrastructure automatically

43. Annotations Used in the Flow

Main Application

```
@SpringBootApplication
```

Controller

```
@RestController  
@RequestMapping  
@GetMapping  
@PostMapping  
@PutMapping  
@DeleteMapping
```

Service

```
@Service
```

Repository

```
@Repository
```

Database Entity

```
@Entity  
@Table  
@Id  
@GeneratedValue
```

Validation

```
@Valid  
@NotBlank  
@Email
```

Transaction

```
@Transactional
```

Exception Handling

```
@RestControllerAdvice  
@ExceptionHandler
```

44. Simple Flow for Students

Remember this first:

```
Client
↓
Tomcat
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
↓
Database
```

And remember:

```
Controller = Request Handling

Service = Business Logic

Repository = Database Operations

Database = Data Storage
```

45. Spring Boot Startup vs Request Flow

These are two different concepts.

Startup Flow

```
main()
↓
SpringApplication.run()
↓
ApplicationContext
↓
```

```
Auto Configuration
↓
Component Scanning
↓
Beans Created
↓
Dependencies Injected
↓
Tomcat Started
↓
Application Ready
```

Request Flow

```
Client
↓
Tomcat
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
↓
Database
```

Students should understand both separately.

46. Real-Time Example

For:

```
GET /employees/101
```

remember:

1. Postman sends GET /employees/101
2. Embedded Tomcat receives request

3. DispatcherServlet receives the request
4. DispatcherServlet finds EmployeeController
5. EmployeeController calls EmployeeService
6. EmployeeService executes business logic
7. EmployeeService calls EmployeeRepository
8. EmployeeRepository uses JPA/Hibernate
9. Database returns employee
10. Repository returns Employee entity
11. Service converts Entity → DTO
12. Controller returns DTO
13. Jackson converts DTO → JSON
14. HTTP response is sent to Postman

47. Interview Answer

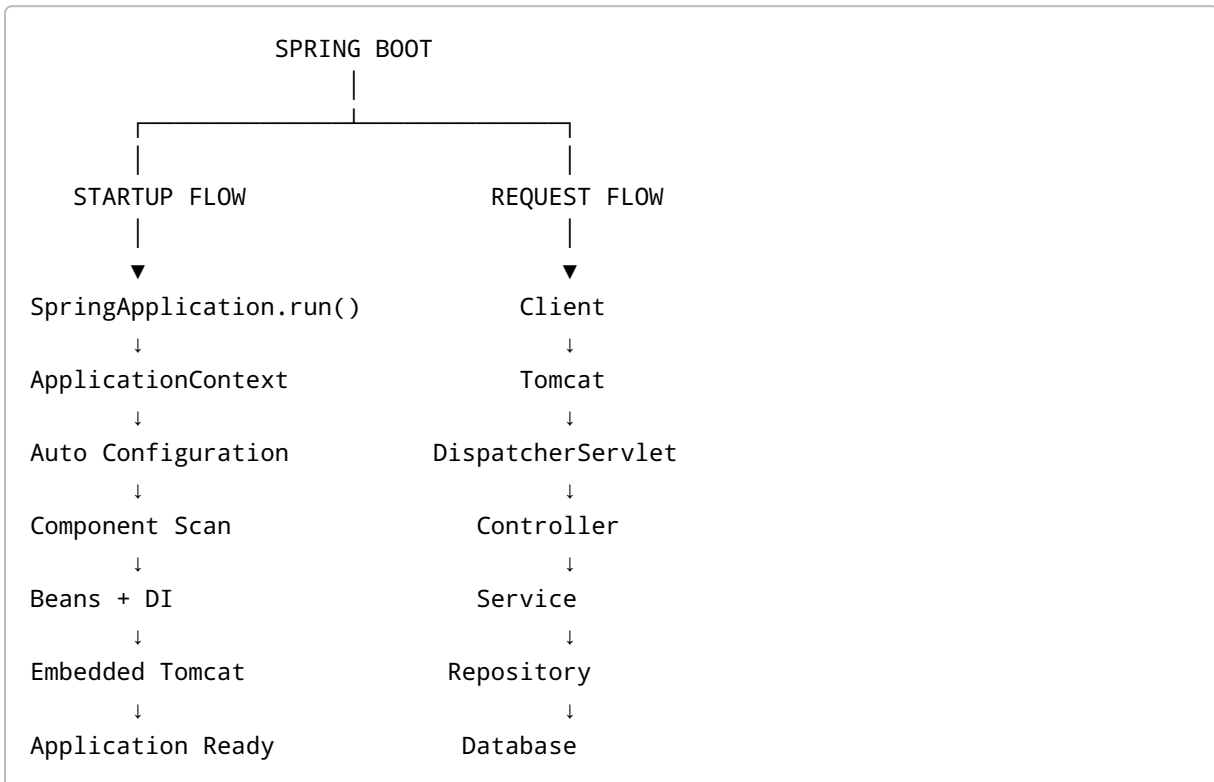
In a Spring Boot REST application, the client sends an HTTP request to the embedded server, usually Tomcat. The request is processed by Spring MVC's DispatcherServlet, which routes it to the appropriate Controller. The Controller calls the Service layer for business logic, and the Service communicates with the Repository for data access. The Repository typically uses Spring Data JPA and Hibernate to interact with the database. The result then travels back through Repository, Service, and Controller, where Spring converts the response object into JSON and returns it to the client. Spring Boot simplifies this architecture through auto configuration, starter dependencies, embedded servers, and externalized configuration.

48. One-Line Interview Answer

Spring Boot request flow is Client → Embedded Server → DispatcherServlet → Controller → Service → Repository → Database, with the response returning through the same layers in reverse.

49. Final Summary

The complete Spring Boot architecture can be remembered as:



The most important point to remember is:

Spring Framework provides the core architecture such as IoC, DI, Spring MVC, DispatcherServlet, and beans, while Spring Boot simplifies the setup by providing auto configuration, starter dependencies, embedded servers, and production-ready features.